

4.5 Algorithm of PrismSpace Multi-Agent System

Input:

- Natural language user objective, task specifications, and operational constraints ($\mathcal{T}_{\text{user}}$).
- Multi-source curated corpora and execution traces ($\mathcal{D}$), spanning LMSYS Arena, RouterBench, WildGuard, CMU Agent Trajectories, and PrismSpace Runtime Events.
- Curated preference alignment pairs ($\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$), containing user prompt x , winning/chosen agent completion y_w , and losing/rejected completion y_l .
- Global multi-agent swarm and tool registry ($\mathcal{G}_{\text{global}} = (V, E)$), comprising the Supervisor Hub, specialized worker agents (Coding, Research, Validator, Planner), and Model Context Protocol (**MCP**: open standard defining structured JSON-RPC communication between agents and tools) endpoints.

Output:

- Predicted intent category ($\hat{y}_{\text{intent}}$) and optimal Large Language Model (**LLM**: deep neural network trained for language comprehension and generation) provider $\hat{y}_{\text{provider}}$ (e.g., Groq, NVIDIA, OpenAI, Anthropic, Google).
- Swarm agent dispatch vector ($\hat{\mathbf{y}}_{\text{agents}} \in \{0, 1\}^{|V|}$) and safety approval gate decision ($\hat{y}_{\text{approval}} \in \{0, 1\}$).
- Active collaboration topology Directed Acyclic Graph ($\mathcal{G}_{\text{active}}$ [**DAG**: a directed graph without closed cycles, establishing execution dependencies]) with cost ($\hat{c}$) and latency ($\hat{\ell}$) predictions.
- Preference-aligned adapter weights ($\Delta \mathbf{W}$) optimizing response quality, policy compliance, and safety margin.
- Performance evaluation metrics: **Accuracy** (correct prediction ratio), **Macro-F1** (unweighted harmonic mean of precision and recall across all classes), **ROC-AUC** (Area Under the Receiver Operating Characteristic Curve measuring discrimination across thresholds), **ECE** (Expected Calibration Error: average gap between confidence and accuracy), **Brier Score** (probability forecast error), R^2 (Coefficient of Determination: explained variance), and **MAE** (Mean Absolute Error: average magnitude of prediction errors).

Step 1: Data Acquisition and Preprocessing

1. Ingest heterogeneous records from multi-source corpora ($\mathcal{D}$) covering intent traces, safety pairs, tool telemetry, and conversational benchmarks.
2. Clean and normalize feature values (strip corrupt 8-bit Unicode Transformation Format [**UTF-8**: standard variable-width character encoding] sequences, normalize whitespace, remove duplicates).
3. Encode categorical agent attributes and tool capabilities (e.g., persona role, execution scope).
4. Standardize all numerical features to zero mean and unit variance:

$$z_i = \frac{x_i - \mu_x}{\sigma_x}$$

5. Split the dataset into 80% training ($\mathcal{D}_{\text{train}}$), 10% validation ($\mathcal{D}_{\text{val}}$), and 10% testing ($\mathcal{D}_{\text{test}}$) partitions.

Step 2: Graph Construction

1. Treat each agent persona and MCP tool v_i as a node in the swarm graph (V).
2. Compute pairwise similarity between feature vectors using **Pearson correlation**:

$$s_{ij} = \frac{\text{Cov}(\mathbf{x}_i, \mathbf{x}_j)}{\sigma_{\mathbf{x}_i} \sigma_{\mathbf{x}_j}}$$

where Cov denotes Covariance (statistical measure of joint directional variance between two feature distributions).

3. Define adjacency matrix $\mathbf{A}$ such that:

$$A_{ij} = \begin{cases} 1, & \text{if } s_{ij} \geq \tau \\ 0, & \text{otherwise} \end{cases}$$

where τ is the similarity threshold.

4. Construct the graph $\mathcal{G} = (V, E)$ with edges E derived from adjacency matrix $\mathbf{A}$.

Step 3: Graph Attention Network (GAT) Initialization

1. Initialize node embeddings $\mathbf{h}_i = \mathbf{x}_i$ for all nodes $v_i \in V$ in the Graph Attention Network (**GAT**: neural architecture utilizing self-attention to weight graph neighbor connections).
2. Initialize weight matrix $\mathbf{W}$ and attention vector $\mathbf{a}$ with random values.
3. Set Learning Rate (**LR**: optimization step size governing parameter updates), number of attention heads (K), and dropout probability (p).

Step 4: Attention-Based Message Passing

1. For each node v_i , compute attention scores for all its neighbors $v_j \in \mathcal{N}(i)$:

$$e_{ij} = \text{LeakyReLU}(\mathbf{a}^T [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j])$$

where LeakyReLU denotes Leaky Rectified Linear Unit (activation function with a small non-zero slope for negative values to prevent dead neurons), and $\parallel$ represents feature concatenation.

2. Normalize attention coefficients using the softmax function:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in \mathcal{N}(i)} \exp(e_{ik})}$$

3. Update the hidden representation of node v_i :

$$\mathbf{h}'_i = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j \right)$$

4. Apply dropout regularization and repeat for all layers.

Step 5: Classification and Loss Computation

1. Feed the final embeddings $\mathbf{h}'_i$ into a fully connected classification layer:

$$\hat{y}_i = \text{Sigmoid}(\mathbf{W}_c \mathbf{h}'_i)$$

2. Compute the Binary Cross-Entropy (**BCE**: loss function quantifying divergence between true binary labels and predicted probabilities) loss:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

3. Backpropagate the loss through the network and update parameters $\theta = \{\mathbf{W}, \mathbf{a}, \mathbf{W}_c\}$ using the Adam optimizer (**Adam**: Adaptive Moment Estimation, an algorithm combining momentum and RMSProp gradient scaling).

Step 6: Preference Alignment and Reward Optimization (ORPO)

1. Ingest curated preference triples $\mathcal{D}_{\text{pref}} = \{(x, y_w, y_l)\}$ comprising user prompt x , winning/chosen agent completion y_w , and losing/rejected completion y_l .
2. Parameterize the base causal language model via Parameter-Efficient Fine-Tuning (**PEFT**: techniques adapting foundation models with minimal trainable weights) using Low-Rank Adaptation (**LoRA**: parameter-efficient fine-tuning decomposing weight updates into low-rank matrices $\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{B} \mathbf{A}$).
3. Compute the Supervised Fine-Tuning (**SFT**: training language models via next-token teacher forcing) Negative Log-Likelihood (**NLL**: loss measuring prediction error of ground-truth token sequences) on the chosen response y_w :

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{|y_w|} \sum_{t=1}^{|y_w|} \log P_{\theta}(y_{w,t} \mid x, y_{w,<t})$$

4. Formulate the length-normalized sequence probability $P_{\theta}(y \mid x) = \left(\prod_{t=1}^{|y|} P_{\theta}(y_t \mid x, y_{<t}) \right)^{\frac{1}{|y|}}$ and define the generative odds of sequence y given prompt x :

$$\text{odds}_{\theta}(y \mid x) = \frac{P_{\theta}(y \mid x)}{1 - P_{\theta}(y \mid x)}$$

5. Compute the Odds Ratio preference loss to penalize disfavored agent behaviors without requiring a reference model:

$$\mathcal{L}_{\text{OR}}(\theta) = -\log \sigma \left(\log \frac{\text{odds}_{\theta}(y_w \mid x)}{\text{odds}_{\theta}(y_l \mid x)} \right)$$

where σ denotes the Sigmoid activation function.

6. Optimize the unified Odds Ratio Preference Optimization (**ORPO**: reference-model-free alignment technique directly penalizing disfavored generation odds during supervised fine-tuning) objective:

$$\mathcal{L}_{\text{ORPO}}(\theta) = \mathcal{L}_{\text{SFT}}(\theta) + \beta \cdot \mathcal{L}_{\text{OR}}(\theta)$$

where β is the preference penalty scaling hyperparameter (e.g., $\beta = 0.1$).

Step 7: Real-Time Runtime Inference, Swarm Execution, and UI Generation

1. **User Input Ingestion:** Capture natural language task objective $\mathcal{T}_{\text{user}}$ submitted via the Next.js User Interface (**UI**: client-facing presentation layer for reactive interaction) and transmit via Hypertext Transfer Protocol (**HTTP**: standard web communication protocol) POST to the FastAPI Hive Bridge Application Programming Interface (**API**: formal programmatic contract enabling service communication).

2. **Query Vectorization:** Extract lexical, semantic, and telemetry features:

$$\mathbf{x}_{\text{query}} = [\mathbf{v}_{\text{TF-IDF}}(\mathcal{T}_{\text{user}}) \parallel \mathbf{e}_{\text{dense}}(\mathcal{T}_{\text{user}}) \parallel \mathbf{f}_{\text{meta}}]$$

where $\parallel$ denotes vector concatenation, and $\mathbf{f}_{\text{meta}}$ encodes query length, token count, and domain complexity.

3. **GAT Model Routing Inference:** Compute hidden query state $\mathbf{h}'_{\text{query}}$ to predict intent, optimal LLM provider, and active agent dispatch mask:

$$\hat{y}_{\text{intent}}, \hat{y}_{\text{provider}} = \text{argmax}(\text{Softmax}(\mathbf{W}_{\text{route}} \mathbf{h}'_{\text{query}})), \quad \hat{\mathbf{y}}_{\text{agents}} = \mathbb{I}(\text{Sigmoid}(\mathbf{W}_{\text{agent}} \mathbf{h}'_{\text{query}}) \geq \tau_{\text{dispatch}})$$

where $\mathbb{I}(\cdot)$ is the indicator function and τ_{dispatch} is the routing activation threshold.

4. **Active DAG Task Scheduling:** Synthesize Directed Acyclic Graph $\mathcal{G}_{\text{active}} = (V_{\text{active}}, E_{\text{active}})$ over active agents $V_{\text{active}} = \{v_i \in V \mid \hat{\mathbf{y}}_{\text{agents},i} = 1\}$, sorted into execution sequence $\pi = \text{TopologicalSort}(\mathcal{G}_{\text{active}})$.
5. **Autonomous ReAct Swarm Execution:** For each agent $v_i \in \pi$, execute Reasoning and Acting (**ReAct**: paradigm interleaving chain-of-thought reasoning with tool execution actions) invoking Model Context Protocol (**MCP**) endpoints via JavaScript Object Notation Remote Procedure Call (**JSON-RPC**: lightweight remote procedure call protocol):

$$a_{i,t} = \text{LLM}_{\hat{y}_{\text{provider}}}(\mathcal{T}_{\text{user}}, h_{i,<t}), \quad o_{i,t} = \text{ExecMCP}(a_{i,t})$$

where $a_{i,t}$ represents the action/tool call payload and $o_{i,t}$ denotes the environment observation.

6. **ORPO Safety Gate Verification:** Evaluate candidate completion $\hat{y}_{\text{candidate}}$ using the preference policy:

$$\text{Score}_{\text{policy}} = \log \frac{\text{odds}_{\theta}(\hat{y}_{\text{candidate}} \mid \mathcal{T}_{\text{user}})}{\text{odds}_{\theta}(y_{\text{ref}} \mid \mathcal{T}_{\text{user}})}$$

Approve output if $\text{Score}_{\text{policy}} \geq \tau_{\text{safety}}$; otherwise reroute to Validator agent for policy correction.

7. **Real-Time Streaming to UI and Client Persistence:** Stream output tokens and agent thought states to the client via Server-Sent Events (**SSE**: persistent unidirectional streaming protocol):

$$\text{State}_{\text{UI}}^{(t)} = \text{State}_{\text{UI}}^{(t-1)} \cup \{\Delta_{\text{token}}^{(t)}, \text{Card}_{\text{agent}}\}$$

Upon stream completion, commit conversation turns, generated artifacts, and telemetry to the browser's local IndexedDB (Dexie.js) for reactive rendering via `useLiveQuery`.

Step 8: Model Evaluation

1. Evaluate the trained routing and alignment models on validation and test partitions.
2. Compute classification and preference evaluation metrics:
 - **Accuracy:** Ratio of correct predictions over total predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

where:

- **TP:** True Positives (positive cases correctly identified as positive).
 - **TN:** True Negatives (negative cases correctly identified as negative).
 - **FP:** False Positives (negative cases incorrectly classified as positive; Type I error).
 - **FN:** False Negatives (positive cases incorrectly classified as negative; Type II error).
 - **AUC:** Area Under the Curve of the Receiver Operating Characteristic (**ROC:** curve plotting True Positive Rate vs. False Positive Rate across thresholds) to assess discrimination capability.
 - **AUPRC:** Area Under the Precision-Recall Curve (metric evaluating precision-recall trade-offs especially under severe class imbalance).
3. Save the trained model parameters, LoRA adapters, and generate prediction outputs.

Step 9: Visualization and Interpretation

- Visualize learned attention weights to identify key routing features contributing to optimal swarm delegation.
- Generate performance plots (loss curve, ROC, precision-recall, reward margin distribution).
- Analyze high-attention edges to reveal biologically meaningful and task-optimal variant relationships.

Algorithm Summary

The algorithm coordinates intelligent model routing, preference alignment, and real-time execution into a cohesive user workflow. By modeling specialized agents and tools through a Graph Attention Network (**GAT**), the system learns how agent capabilities complement one another, dynamically routing each incoming request to the most capable and cost-effective language model provider. To guarantee that generated actions remain safe and compliant with user intentions, Odds Ratio Preference Optimization (**ORPO**) directly refines model behaviors against curated preference pairs without reference model overhead. Once an objective is submitted in the web interface, the algorithm deconstructs it into a structured plan, orchestrates autonomous agents to execute tools, verifies safety policies, and streams responses incrementally into the browser's local IndexedDB (Dexie.js). This design ensures transparent, dependable, and responsive multi-agent task execution across both research workflows and enterprise applications.